FundleBrain

Loyalty, CRM & Customer-Intelligence Platform

Technical & Architecture Overview

Prepared for	Client technical / code review
Document date	02 September 2026
Scope	Technology stack, AI/LLM usage, integrations, security, code quality & QA
Classification	Confidential — for the intended recipient only

This document describes the actual, implemented state of the platform as verified against the source code and running services. It intentionally avoids forward-looking claims except where a section is explicitly labelled “Roadmap”. Software/library version numbers are quoted from the project dependency manifests.

fundle

1. Executive Summary

The platform is a single-tenant, enterprise-grade loyalty and customer-intelligence system for the retail brand. It unifies point-of-sale loyalty processing, a CRM/customer 360, analytics and reporting, campaign automation (SMS), an AI analytics assistant, support tooling and API monitoring.

- **Architecture:** a Python/FastAPI asynchronous REST backend, a React single-page frontend, and MongoDB (managed replica set) as the primary data store.
- **Scale:** the production data store holds a large customer base (~1M+ customer records) and a high-volume transaction / points ledger, served through indexed, read-preference-tuned queries.
- **AI:** a conversational, action-enabled analytics assistant backed by frontier LLMs (Anthropic Claude, OpenAI GPT, Google Gemini), accessed via LLM provider API keys held only in the cloud environment — no provider keys are stored in the application source.
- **Messaging:** DLT-compliant transactional and bulk marketing SMS via the Karix (Tanla) gateway, with delivery-receipt tracking.
- **Security:** JWT + bcrypt authentication, ten-role RBAC, portal gating, full audit logging, environment-only secrets and CORS allow-listing.

2. System Architecture

A conventional, well-separated three-tier architecture. All backend routes are namespaced under **/api** and served by an ASGI stack; the frontend is a compiled SPA that talks to the backend exclusively over HTTPS using a configurable base URL. Long-running and scheduled work is handled by an in-process async scheduler and background tasks rather than blocking request threads.

Tier	Responsibility	Core technology
Presentation	Admin CRM dashboard, store portal, reports, campaign manager, AI console	React 19 SPA (Tailwind CSS + Radix UI / shadcn pattern)
Application / API	REST endpoints, business logic, auth, AI orchestration, messaging	FastAPI (ASGI) on Uvicorn, 58 modular route packages
Data	Customers, transactions, points ledger, campaigns, audit & API logs	MongoDB 7.0 replica set (via async Motor driver)
Async / jobs	Weekly digests, report ingestion, automated campaigns, daily reconciliation	APScheduler (AsyncIO, Asia/Kolkata) + FastAPI background tasks

Cross-cutting middleware: a CORS policy (explicit allow-list plus a domain regex) and a pure-ASGI API-logging middleware that records every request's method, path, status and latency for the built-in API monitor (kept as raw ASGI specifically so streaming CSV/XLSX/PDF exports are never truncated).

3. Backend Technology Stack

Component	Technology	Version
Language / runtime	Python	3.11
Web framework	FastAPI	0.110.1
ASGI server	Uvicorn	0.25.0
ASGI toolkit	Starlette	0.37.2
Data validation / models	Pydantic	2.13.4
MongoDB driver (async)	Motor / PyMongo	3.3.1 / 4.5.0
Auth — JWT	PyJWT	2.12.1
Auth — password hashing	bcrypt	4.1.3
Scheduler	APScheduler	3.11.2
Data processing	pandas / numpy	3.0.3 / 2.4.4
Excel / PDF generation	openpyxl / reportlab	3.1.5 / 4.5.1
LLM orchestration	LiteLLM (OpenAI-compatible gateway)	1.80.0
HTTP client	httpx / requests	0.28.1 / 2.34.1

4. Frontend Technology Stack

Component	Technology	Version
UI library	React	19.0
Routing	React Router DOM	7.5
Build tooling	Create React App + CRACO	5.0.1 / 7.1
Styling	Tailwind CSS	3.4
Component primitives	Radix UI (shadcn/ui pattern)	1.x–2.x
Charts / dataviz	Recharts	3.6
Animation	Framer Motion	12.42
HTTP client	Axios	1.8
Forms & validation	React Hook Form + Zod	7.56 / 3.24
Notifications	Sonner (toasts)	2.0
Icons	lucide-react	0.507
Package manager	Yarn	1.22

fundle

5. Data Layer — MongoDB

The primary datastore is MongoDB 7.0, running as a managed, multi-node replica set in production. Access is fully asynchronous via the Motor driver.

Aspect	Detail
Server version	MongoDB 7.0 (managed cloud replica set; development verified on 7.0.42)
Connectivity	Connection pooling (max pool 100), automatic retryable reads & writes
Write concern	w = majority (durable, acknowledged by a majority of nodes)
Read preference	primaryPreferred — heavy analytics reads tolerate secondaries so primary stays responsive under load
Schema approach	Document models with a shared base that maps Mongo _id → string id; ObjectId is never returned raw to clients (JSON-safe boundaries)
Indexing	~43 managed indexes ensured at startup across the core collections
Data volume (prod)	~50 collections; ~1M+ customer records plus high-volume transaction, points-ledger and API-log collections

6. Artificial Intelligence & LLMs

AI features are delivered through **LLM provider API keys** routed via an OpenAI-compatible LLM gateway (LiteLLM). Consequently **no OpenAI / Anthropic / Google secrets live in the application source or repository** — the keys are injected from the cloud environment. The following models are configured in the code today:

Capability	Provider	Model (as configured)
Analytics assistant — default (“Fundle Brain”)	Anthropic	Claude Sonnet 4.6
Assistant — deep reasoning (“Master Brain”)	Anthropic	Claude Opus 4.7 (selectable)
Assistant — alternate models (user-selectable)	OpenAI / Google	GPT-5.5 / Gemini 3.1 Pro
Campaign copy & report drill-down narratives	OpenAI	GPT-5.2
Uploaded-report narrative summaries	OpenAI	GPT-5
Text-to-speech (product walkthrough audio)	OpenAI	tts-1 / tts-1-hd

Action-enabled assistant: the assistant supports function-calling / tool-use. Data-mutating tools are role-gated, run through a preview → confirm → audited-execute pattern, and are reversible. All AI requests carry an application identifier for usage attribution and cost control.

7. Third-Party Integrations

Only the integrations below are actually wired into the running system. (For transparency: some SDKs present in the dependency list — e.g. object-storage and payment SDKs — are not currently used by any code path, so they are deliberately excluded here.)

Integration	Purpose	Notes
Karix / Tanla (messaging)	DLT-compliant transactional & bulk marketing SMS; WhatsApp send; OTP delivery	Dedicated DLT-registered sender ID; delivery receipts (DLR) received by webhook; egress IP whitelisted with the provider; API keys held only in environment config
Microsoft 365 mailbox (IMAP)	Optional daily auto-pull of the sales reconciliation file	Enabled only when IMAP credentials are supplied via environment; no secrets in code
LLM providers (API keys / MCP)	Gateway to Anthropic / OpenAI / Google models	OpenAI-compatible LLM gateway; keys injected from the cloud environment

8. Security & Access Control

- **Authentication:** stateless JWT (HS256, 12-hour expiry) issued on login; accepted as a Bearer token or an HttpOnly cookie. Passwords are hashed with **bcrypt** (per-password salt); plaintext passwords are never stored or logged.
- **Authorisation (RBAC):** ten distinct roles (super admin, brand admin, CRM manager, marketing manager, regional manager, store manager, store staff, support agent, analytics viewer, read-only executive). Endpoints are guarded by explicit role requirements, and login is additionally **portal-gated** (store portal vs. CRM/admin portal).
- **Audit trail:** logins, logouts and all data-mutating actions are written to an immutable audit log with the actor, action, entity, metadata, IP and timestamp.
- **API monitoring:** every API call is logged (method, path, status, latency) for observability and abuse detection via the built-in API monitor.
- **Secrets management:** all credentials, connection strings, keys and JWT secret come from environment variables only — the codebase contains no hardcoded secrets; missing config fails fast rather than falling back to defaults.
- **Network:** HTTPS/TLS transport end-to-end; the managed cloud database provides encryption in transit and at rest plus an IP access-list; the messaging provider egress IP is whitelisted.
- **CORS:** requests are restricted to an explicit origin allow-list plus a controlled domain regex.
- **Least-privilege demo mode:** public demo accounts are enforced read-only at the auth layer — every write method is rejected except a tiny, explicit allow-list.

Not yet in place (candid disclosure): multi-factor authentication, automated login brute-force lockout, and a formal third-party penetration test / SOC 2 attestation are not currently implemented — these are natural next steps and are noted on the roadmap.

9. Code Quality & Engineering Practices

- **Modular structure:** the backend is split into 58 focused route modules by domain (loyalty, CRM, reports, campaigns, communications, AI, support, monitoring, etc.) rather than a monolith, with shared utilities for money/date normalisation, DB timeouts and serialisation.
- **Typed contracts:** Pydantic v2 models validate all request/response payloads at the API boundary; a shared document base guarantees JSON-safe serialisation from MongoDB.
- **Backend quality tooling:** configured with **black** (formatting), **isort** (imports), **flake8** (lint), and **mypy** (static type checking).
- **Frontend quality tooling: ESLint** with the React, React-Hooks, JSX-a11y and import plugins; consistent component conventions and a shared design system (Tailwind + Radix/shadcn).
- **Automated pre-deploy gate:** a static analysis pass blocks common production hazards (hardcoded secrets, unsafe DB serialisation, wrong ports/URLs, compile errors) before release.
- **Testability:** stable, descriptive test identifiers on interactive UI elements enable reliable end-to-end automation; the API is scriptable and self-documented via FastAPI's OpenAPI schema.

10. Testing & Quality Assurance

- **Backend tests:** pytest is set up with a dedicated test package for API/integration checks.
- **End-to-end QA:** features are validated through an iterative automated UI+API test process; each cycle produces a structured pass/fail report that is remediated before sign-off (100+ recorded iterations to date).
- **Verification discipline:** backend changes are verified with live API calls (real request / response assertions) and, where relevant, direct data checks; UI changes are verified against the running application.
- **Change safety:** the codebase is version-controlled with checkpointing after each change, allowing rollback to any previous known-good state.

11. Deployment & Operations

- **Environments:** a clean separation between a development environment and the live production deployment on cloud infrastructure, each with its own database and configuration.
- **Process management:** backend and frontend run as supervised services; the API is stateless and horizontally scalable behind the ingress.
- **Configuration:** environment-driven (URLs, database, keys, feature flags) with no build-time secrets; the production database is a managed cloud MongoDB cluster.
- **Observability:** built-in health endpoints, the request-level API monitor, application logs, and a background-job scheduler with per-job logging.

12. Roadmap (planned — not yet built)

Listed separately and explicitly so nothing above is mistaken for current capability:

- Meta / WhatsApp Cloud API and Meta Ads custom-audience integration (pending provider credentials).
- Location-wise DLT SMS configuration, reward brands / gift-voucher rewards, and expanded OTP audit search.
- Variable (parameterised) DLT SMS templates for true per-customer personalised marketing copy.

- Security hardening: MFA, login brute-force lockout, and an external security assessment.
- Scheduled backups to object storage and a drag-and-drop report builder.

Prepared from a direct review of the application source code and running services. All version numbers reflect the project dependency manifests at the document date; model identifiers reflect the values configured in code and may be updated as newer managed models become available.